

INDIAN INSTITUTE OF TECHNOLOGY DELHI

COL 729 Lab 3: Studying Compiler Passes

Submitted by

Aditya Anand and Varun Subramaniam
2023CS50284 and 2023CS50497

Instructor

Prof. Sorav Bansal

4 October 2026

1 Part A: Transformation Pass

1.1 Design

`strength-reduce` is a function pass for the new pass manager. It replaces a multiply by a power of two, 2^k, with a left shift by k. The constant can be on either side and the type stays the same. This works because $x \cdot 2^k = x \ll k$ modulo 2^N. Multiplies by 1 and vector multiplies are skipped.

The `nuw` and `nsw` flags are copied to the `shl` since they mean the same on both instructions. No new flag is added because that could create a `poison` value. If the pass changes nothing, it returns `PreservedAnalyses::all()` otherwise it preserves the CFG analyses since no blocks are touched.

1.2 Testing

I ran the pass on my test cases in `input.ll` and checked the output with LLVM's verifier. The table lists each case and what it should become.

Case	Multiply	Expected
Right constant, i32	`mul i32 %x, 8`	`shl i32 %x, 3`
Left constant, i32	`mul i32 16, %x`	`shl i32 %x, 4`
Right constant, i64, nsw	`mul nsw i64 %x, 64`	`shl nsw i64 %x, 6`
Left constant, i64, nuw	`mul nuw i64 1024, %x`	`shl nuw i64 %x, 10`
One	`mul i32 %x, 1`	unchanged
Zero	`mul i32 %x, 0`	unchanged
Negative constant	`mul i32 %x, -4`	unchanged
Non power of two	`mul i32 %x, 3`	unchanged
Vector value	`mul <4 x i32> %x, <i32 4, i32 4, i32 4, i32 4>`	unchanged

1.3 Results

I built the plugin, ran it on `input.ll`, checked the output with the verifier and compared the program output before and after (Figure 1). The four positive cases became `shl` and the five negative cases are unchanged. The outputs are identical.

```
● → Part A git:(main) ✗ export PATH="/opt/homebrew/opt/llvm@18/bin:$PATH"
● → Part A git:(main) ✗ clang++ -fPIC -shared -undefined dynamic_lookup \
  -o libStrengthReduce.so part1.cpp \
  $(llvm-config --cxxflags)
● → Part A git:(main) ✗ opt -load-pass-plugin ./libStrengthReduce.so \
  -passes=strength-reduce -S input.ll -o output.ll
● → Part A git:(main) ✗ opt -passes=verify -disable-output output.ll
● → Part A git:(main) ✗ ./compare.sh
BEFORE (input.ll)                          | AFTER (output.ll)
-------------------------------------------+-------------------------------------------
mul i32 7, 8                            56 | shl i32 7, 3                            56
mul i32 16, -3                         -48 | shl i32 -3, 4                          -48
mul i64 -1234567, 64           -79012288 | shl nsw i64 -1234567, 6        -79012288
mul i64 1024, 999                1022976 | shl nuw i64 999, 10               1022976
mul i32 7, 0                           0 | mul i32 7, 0                           0
mul i32 7, 1                           7 | mul i32 7, 1                           7
mul i32 7, 3                          21 | mul i32 7, 3                          21
mul i32 7, -4                         -28 | mul i32 7, -4                         -28
mul vec 1,2,3,4 * 4       <4, 8, 12, 16> | mul vec 1,2,3,4 * 4       <4, 8, 12, 16>
-------------------------------------------+-------------------------------------------
No difference
```

Figure 1: Before and after outputs for every case.

1.4 Limitations

A multiply of two constants (`mul i32 8, 4`) becomes `shl i32 8, 2` instead of being folded, though `instsimplify` would clean that up. Also, a power of two whose top bit is set such as `INT_MIN` (-2^{31} in `i32`) reads as negative and is skipped, so it stays as a `mul`.

2 Part B: Favourite analysis: definitive initialization

This is the analysis that I found most interesting, and it is my favourite, because I can think of further applications of this type of analysis. For example, when there are many variables and the registers cannot hold all of them, the variables that are *maybe uninitialized* can be spilled to memory without sacrificing too much of the program's performance.

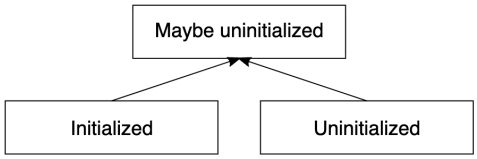
Example: definitive initialization

Definitive initialization proves that variables are known to be initialized when read. If we find a variable which is read when not initialized then we generate a warning.

```
void Init() {
  int x;    // x is uninitialized
  if (cond()) {
    x = 10; // x is initialized
  } else {
    x = 20; // x is initialized
  }
  print(x); // x is initialized
}
```

```
void Uninit() {
  int x;    // x is uninitialized
  if (cond()) {
    x = 10; // x is initialized
  }
  print(x); // x is maybe uninitialized, x is being read, report a bug.
}
```

For this purpose we can use lattice in a form of a mapping from variable declarations to initialization states; each initialization state is represented by the following lattice:



A lattice element could also capture the source locations of the branches that lead us to the corresponding program point. Diagnostics would use this information to show a sample buggy code path to the user.

2. Proposed analysis: use-after-free (segfault) analysis

One more analysis worth implementing, similar to definitive initialization, is a use-after-free (segfault) analysis. At every program point, the analysis keeps the set of heap pointers that *must* already have been freed along all paths reaching that point, or $\top$ if the point has not been reached.

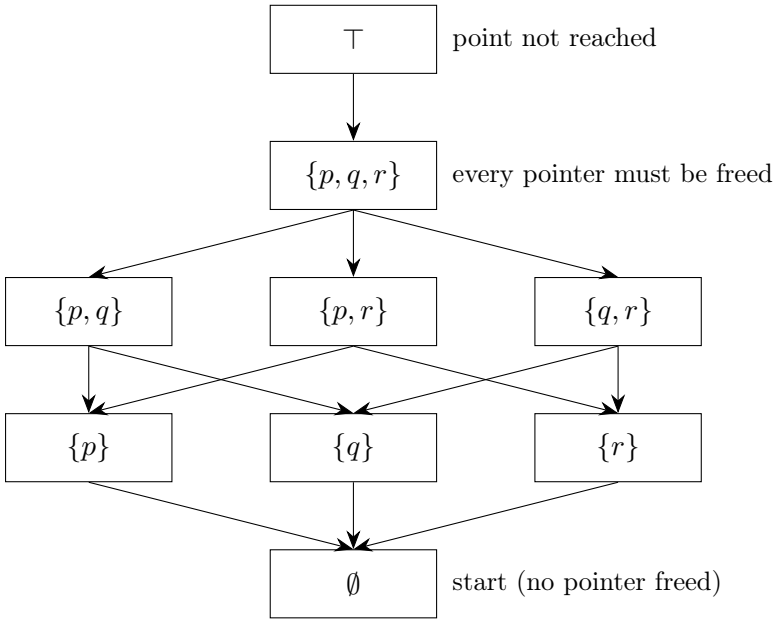
- **Direction:** forward.
- **Domain:** $2^P \cup \{\top\}$, where P is the set of heap pointers in the function, ordered by $\subseteq$. The value $\top$ is above every set and means that this program point has not been reached.
- **Initial value:** $\emptyset$ at the function entry, since nothing has been freed yet, and $\top$ everywhere else.
- **Meet (merge) operator:** intersection, with $\top \sqcap X = X$. If a pointer is freed on all incoming path, its usage is definitely unsafe.

- **Transfer functions:** ($f(\top) = \top$, otherwise)

$$\begin{aligned} \texttt{free(p)}: &\quad OUT = IN \cup \{p\} \\ \texttt{p = malloc(...)}: &\quad OUT = IN \setminus \{p\} \\ \text{otherwise}: &\quad OUT = IN \end{aligned}$$

- **Check:** if `p` is dereferenced (or freed again) at a point where $p \in IN$, report a definite use-after-free (or double free).

For three heap pointers $P = \{p, q, r\}$, the lattice is the following (the analysis starts at $\emptyset$):




```
void UseAfterFree() {
  int *p = malloc(sizeof(int)); // freed = {}
  if (cond()) {
    free(p);                    // freed = {p}
  } else {
    free(p);                    // freed = {p}
  }
  *p = 5; // freed = {p} n {p} = {p}, p must be freed, report a bug.
}
```

Talking about generality, payoff, and ease-of-implementation, this follows a very similar DFA to the other analysis studied in the class

This analysis can be used to prevent memory from being accessed after it has been freed, catching such bugs at compile time instead of as a segmentation fault at run time.

3 Part C

3.1 Snippets And The Generated CFG

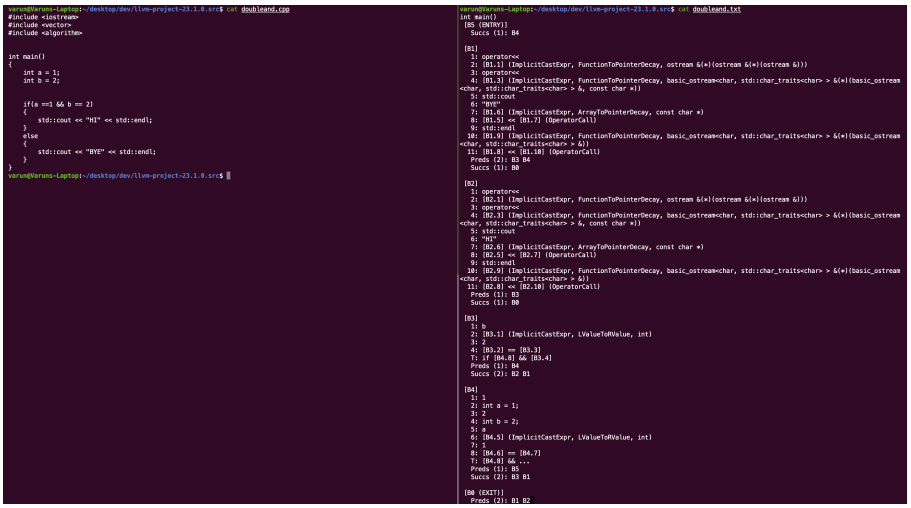


Figure 2: Snippet using &&

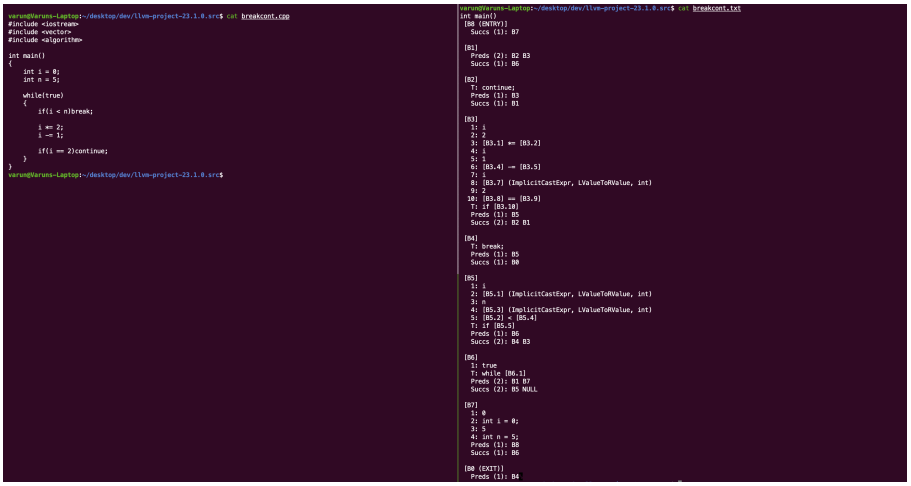


Figure 3: Snippet using break and continue statements

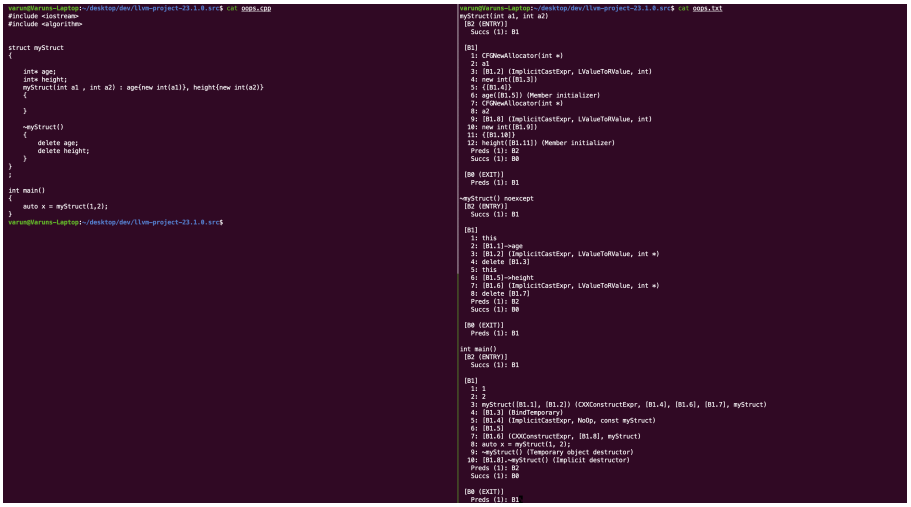


Figure 4: Snippet using constructor and destructor

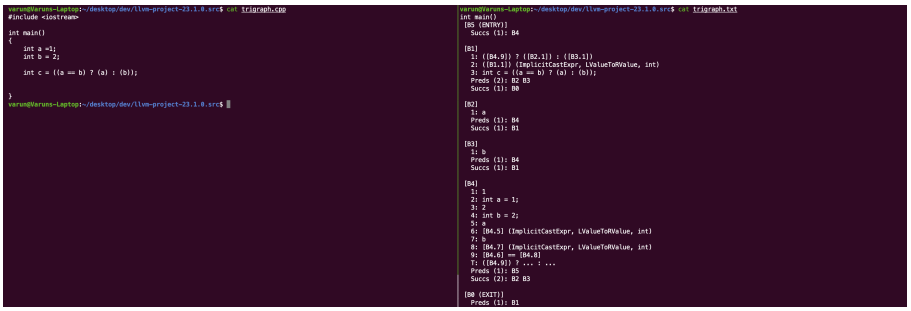


Figure 5: Snippet using ternary expressions

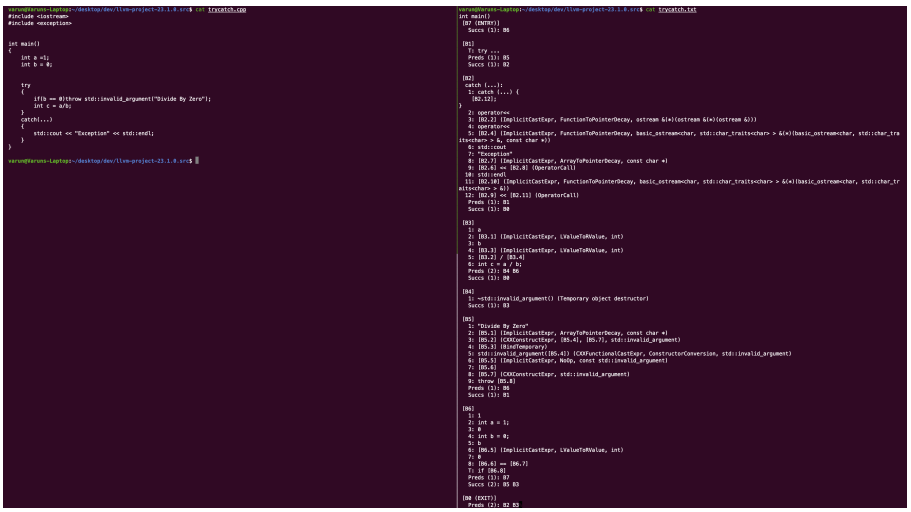


Figure 6: Snippet using try-catch block

3.2 Observations and Conclusions

3.2.1 Ternary Operator Analysis

A ternary operator's test term is part of the current basic block, but the calculations of the 2 possible values is done as a new basic block. This is seen in the ternary operator code snippet as well. This is non-obvious because a phi node could also be expected in the case of ternary operators.

VisitConditionalOperator evaluates the condition in the current block, with ?: as its terminator. The true and false expressions each get their own block

3.2.2 && Analysis

The analysis of && shows that the evaluation of the first condition is done in the current basic block(B4 in the given example) and the evaluation of the other condition is done in a new basic block (B3). This is non-obvious because another possible way of analysis is calculating the condiional in the same basic block.

VisitLogicalOperator evaluates the LHS in the current block and gives the RHS its own block. The LHS block ends with the && as its terminator, with two successors: the RHS block if the LHS is true, and the join block if it is false.

3.2.3 Break and Continue Analysis

The main calculation of the loop is handled in B3. However the statement for the break condition corresponds to a new basic block(B5) in the snippet. However, the conditional check for continue (which comes at the end of the loop) is a part of B3 itself instead of creating a new basic block for it. This is non-obvious because break is given a separate basic block but continue is not.

3.2.4 Try-Catch Block Analysis

The try block does not get its own basic block. It is part of the execution of main() itself. This is somewhat counter-intuitive. After that, there is a separate basic block for creating and throwing an exception (B5) and a separate basic block for catch (B2). The corresponding block numbers can be seen in the snippets.

3.2.5 Constructor and Destructor Analysis

The constructor and destructor code is made unrelated in the CFG generated. myStruct() and m̃yStruct() are seen as completely unrelated functions in the CFG with each having different entry and exit points. main() also has its own entry and exit points.

Since I am using {} initialization, the new int() are called with the input parameters itself and not populated later.

We can also see a temporary being constructed in the main() function and then this being copied to auto x. This is pre C++17 behaviour.

These are some interesting behaviours seen in the code snippet dump.

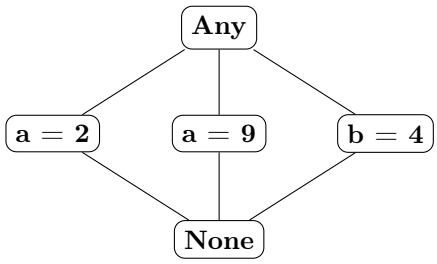
4 Part D: Single Variable Constant Propagation

4.1 Lattice

The lattice element is the struct **ConstantPropagationLattice**:

```
struct ConstantPropagationLattice {
  struct VarValue {
    const VarDecl *Var;
    int64_t Value;
    friend bool operator==(VarValue Lhs, VarValue Rhs) {
      return Lhs.Var == Rhs.Var && Lhs.Value == Rhs.Value;
    }
  };
  // ‘std::nullopt‘ is "bottom".
  std::optional<VarValue> Data;
};
```

It has three kinds of elements:



top(): Data = {Var = nullptr, Value = 0}

Data = VarValue{Var, Value}

bottom(): Data = std::nullopt

None is the bottom where nothing is known yet. A middle element says one variable has one fixed value. **Any** is the top where the value is not a single constant. The analysis starts at **None** and the lattice has three levels.

When two paths meet, **join()** keeps the element if both paths agree and gives **Any** otherwise, for example **a = 2** with **a = 9**. Joining **None** into an element changes nothing and joining an element into **None** gives that element.

4.2 Transfer function

transfer() looks at one statement at a time and sets the new element from what that statement does. Anything that is not a statement is ignored.

```
transfer(E, Element):
  if E is not a CFGStmt:
      return

  if E is "int x = e" or "x = e":    // declaration or assignment
      if e is a constant c:
          Element = (x, c)
      else:
          Element = Any

  else if E is "x op= e":            // op is +, -, *, and so on
      Element = Any

  else:
      return
```

When a statement matches, the old element is replaced and not joined with the new one. The element holds only one variable so the latest declaration or assignment takes over. A constant

right hand side such as **2 + 3** gives **x = 5**, while **x = g()** gives **Any**. For example, the code

```
int a = 4;
a = 6;
a *= 2;
```

goes through **None**, **a = 4**, **a = 6**, and then **Any**. The last step gives up because compound assignments are never evaluated even when the result would be easy to compute.

4.3 LatticeJoinEffect

LatticeJoinEffect is the return type of **join()**. Its two values, **Changed** and **Unchanged**, tells us whether the join modified the element it was called on. In this analysis, **join()** behaves as follows:

Element	Joined with	Result	Effect
a = 2	**a = 2**	**a = 2**	**Unchanged**
a = 2	**None**	**a = 2**	**Unchanged**
Any	anything	**Any**	**Unchanged**
None	**a = 2**	**a = 2**	**Changed**
a = 2	**a = 9**	**Any**	**Changed**

In the first three rows the element already covers the incoming one so nothing moves. In the last two it goes up the lattice.

The fixed point algorithm uses this value to decide whether it has to keep going or not. When a block's state is **Changed**, the blocks after it may be affected so they are visited again. When it is **Unchanged**, the block has settled. So the two values tell the algorithm when to repeat and when to stop. Once no block changes, the analysis has reached its fixed point and since elements only move up a lattice of three levels, this always happens.